

Let the Trial Begin: A Mock-Court Approach to Vulnerability Detection using LLM-Based Agents

Ratnadira Widyasari

Singapore Management University
Singapore, Singapore
ratnadiraw.2020@phdcs.smu.edu.sg

Martin Weyssow

Singapore Management University
Singapore, Singapore
mweyssow@smu.edu.sg

Ivana Clairine Irsan

Singapore Management University
Singapore, Singapore
ivanairsan@smu.edu.sg

Han Wei Ang

GovTech
Singapore, Singapore
ang_han_wei@tech.gov.sg

Frank Liauw

GovTech
Singapore, Singapore
frank_liauw@tech.gov.sg

Eng Lieh Ouh

Singapore Management University
Singapore, Singapore
elouh@smu.edu.sg

Lwin Khin Shar

Singapore Management University
Singapore, Singapore
lkshar@smu.edu.sg

Hong Jin Kang

University of Sydney
Sydney, Australia
hongjin.kang@sydney.edu.au

David Lo

Singapore Management University
Singapore, Singapore
davidlo@smu.edu.sg

Abstract

Detecting vulnerabilities in source code remains a critical challenge, especially when benign and vulnerable functions share significant similarities. We introduce VULTRIAL, a courtroom-inspired multi-agent framework designed to identify vulnerable code and to provide explanations. It employs four role-specific agents, *security researcher*, *code author*, *moderator*, and *review board*. Using GPT-4o as the base LLM, VULTRIAL almost doubles the efficacy of prior best-performing baselines, with further improvement from role-specific instruction tuning using limited data. Our extensive experiments demonstrate the efficacy of VULTRIAL across different LLMs, including an open-source, in-house-deployable model (LLaMA-3.1-8B), as well as the high quality of its generated explanations and its ability to uncover multiple confirmed zero-day vulnerabilities in the wild.

Keywords

vulnerability detection, large language models, multi-agent

ACM Reference Format:

Ratnadira Widyasari, Martin Weyssow, Ivana Clairine Irsan, Han Wei Ang, Frank Liauw, Eng Lieh Ouh, Lwin Khin Shar, Hong Jin Kang, and David Lo. 2026. Let the Trial Begin: A Mock-Court Approach to Vulnerability Detection using LLM-Based Agents. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3744916.3773256>

1 Introduction

Vulnerability detection in software code is a critical task to ensure system security and prevent potential exploits [7]. Manual vulnerability detection can be time-consuming and error-prone, especially

given the scale and complexity of modern software systems. To automate this, researchers have explored various machine learning and deep learning approaches [4, 9, 42, 43, 71, 72]. Recently, with substantial advancements in large language models (LLMs) and their impressive performance on natural language tasks [1, 35, 54, 70], several studies have found promising outcomes using LLMs for automated vulnerability detection [37, 50, 52, 59, 65, 69].

Despite these advances, recent research by Ding et al. [12] highlighted significant limitations when evaluating current LLM-based vulnerability detection approaches. Their study proposed a challenging new benchmark called PrimeVul, where the functional differences between benign and vulnerable code pairs are subtle, less than 20%. These cases where the code has a high degree of similarity are critical to evaluate because in real-world scenarios, vulnerabilities often stem from subtle, non-obvious code changes [36, 51]. On this dataset, LLM-based approaches exhibited low performance. The best performance comes from using GPT-4 [39], which achieves a Pair-wise Correct Prediction (P-C) score of 40 (out of a possible 435). P-C measures how many vulnerable-benign code pairs a method classifies entirely correctly, directly reflecting its ability to detect the subtle code changes that introduce a vulnerability. This LLM-based approach used in PrimeVul was implemented under the conventional single-agent system, wherein one model attempts to identify and explain vulnerabilities on its own. Single-agent systems are inherently constrained by their reliance on a single perspective when analyzing vulnerabilities, which can lead to issues such as hallucination and overconfidence in incorrect reasoning [14, 28, 31, 45, 66]. These single-agent systems may fail to critically verify their own output, which is crucial when identifying subtle vulnerabilities [31].

Given these limitations, recent research has begun exploring multi-agent systems to automate software engineering tasks [24, 26, 57]. Such approaches distribute tasks across role-specific agents while allowing discussion and richer information exchange between agents. In this work, we propose VULTRIAL, a novel multi-agent LLM framework that employs a courtroom-inspired scenario. VULTRIAL features four role-specific agents: the *security researcher* (prosecutor), *code author* (defense attorney), *moderator* (judge), and



This work is licensed under a Creative Commons Attribution 4.0 International License. ICSE '26, Rio de Janeiro, Brazil

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2025-3/26/04
<https://doi.org/10.1145/3744916.3773256>

review board (jury). The *review board* bases its decision on arguments from the *security researcher* (highlighting potential vulnerabilities), the *code author* (defending the code), and the *moderator* (summarizing debate points). The *review board agent* works similarly to a jury in the courtroom scenario, where they need to hear all the evidence presented in the case and make a final verdict based on it. In VULTRIAL, we use the same underlying LLM for all agents, building on prior works [5, 26, 56] that show prompt-induced roles can lead to different reasoning paths using the same model.

We benchmarked VULTRIAL against single-agent and multi-agent baselines. For the single-agent, we include deep learning (DL)-based models (CodeBERT [16], CodeT5 [55], UniXCoder [21], and LineVul [18]), as well as an LLM-based method (Ding et al.'s CoT prompts [12]). For the multi-agent setting, we utilized GPTLens [25]. All LLM-based methods are instantiated using GPT-3.5 and GPT-4o as the base model (see Section 3.3). The results show that VULTRIAL improves over the baselines. With GPT-4o, VULTRIAL nearly doubles the performance of both the best performing single-agent and multi-agent baselines (P-C score=81 vs. 40 and 44, respectively).

To assess the effect of instruction tuning on role-specific agents, we tuned each agent in VULTRIAL using 50 validation sample pairs. Instruction tuning the *moderator agent* in VULTRIAL improves the P-C score by 15 (18.52%) over the non-tuned version and surpasses the single-agent and multi-agent baselines by 56 (140%) and 52 (118%). To assess generalizability, we additionally confirmed the efficacy of VULTRIAL using LLaMA-3.1-8B, a publicly available model that can be deployed in-house. Separately, we perform an ablation study to verify that each agent contributes positively to achieving the best performance. Beyond quantitative results, we also qualitatively evaluate the generated explanations, examining completeness, clarity, actionability, and informativeness, and observe strong performance across all four dimensions. Finally, we deploy VULTRIAL in the wild, where it successfully uncovers multiple confirmed zero-day vulnerabilities, demonstrating its practical effectiveness.

To summarize, the contributions of our study are as follows:

- We propose **VULTRIAL**, a novel multi-agent, courtroom-inspired approach leveraging LLMs for automated vulnerability detection. Also, different from many prior approaches that only output a binary label, VULTRIAL provides *detailed explanations* for each classification.
- We demonstrate that targeted instruction tuning of role-specific agents in VULTRIAL improves the performance.
- We conduct extensive experiments demonstrating the efficacy of VULTRIAL on benchmark data and in the wild.

2 VULTRIAL

VULTRIAL leverages multiple LLM-based agents modeled after courtroom roles: a *security researcher* (prosecutor), *code author* (defense attorney), *moderator* (judge), and *review board* (jury). This structured debate enables thorough, adversarial discussions for deeper analyses and refined conclusions regarding code vulnerabilities. Drawing inspiration from courtroom practices, where structured adversarial processes have been shown to reduce wrongful convictions and improve the fairness of judgments [17, 20], VULTRIAL aims to improve vulnerability detection in a similar manner.

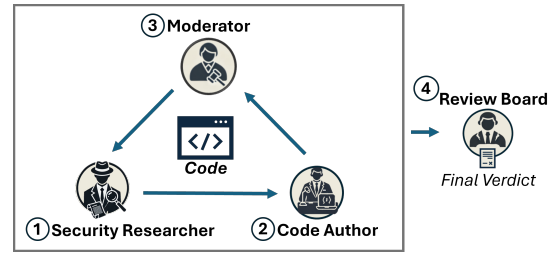


Figure 1: VULTRIAL architecture which uses a courtroom-inspired multi-agent framework where the *security researcher* (prosecutor) identifies potential vulnerabilities, the *code author* (defense attorney) provides counterarguments, the *moderator* (judge) summarizes key points, and the *review board* (jury) delivers the final verdict on code vulnerabilities.

VULTRIAL architecture is shown in Fig. 1, with an example output in Fig. 2. The first agent acts as a *security researcher* and is responsible to analyze the given code and identify potential vulnerabilities. The second agent, namely the *code author agent*, aims to provide counterarguments and justifications to refute the claims made by the *security researcher agent*. The third agent, which is the *moderator agent*, is tasked with summarizing the arguments presented by both the *security researcher* and the *code author agents* to ensure a structured exchange of viewpoints between them. VULTRIAL supports multiple rounds of interaction among the first three agents before passing their viewpoints to the last agent. The last agent, namely the *review board agent*, is tasked with analyzing all the information given by the *moderator*, the *security researcher*, and the *code author agents*. Based on this collective information, the *review board* delivers a final verdict regarding the security status of the code. We chose to have the agents simulate a jury over a judge sentencing scenario, as previous studies have shown that jury participation can help prevent excessively harsh punishments [22, 34]. The details of the agent's interaction will be discussed in the following subsections. The prompts can be found in our online appendix [60].

2.1 Security Researcher Agent (Prosecutor Role)

In VULTRIAL, the *security researcher agent* functions similarly to a courtroom prosecutor. A prosecutor's role is to investigate evidence, establish the facts, and build a case against a defendant (i.e., the party that is accused in a court of law). For vulnerability detection, the *security researcher agent* inspects the provided code, identifies potential vulnerabilities, and explains why they may pose a threat.

In the first round, the agent receives a code function and determines whether the code is vulnerable. In subsequent rounds, it also receives the output generated by two other agents (i.e., *code author* and *moderator*). Beyond simply reporting any vulnerability, the agent is explicitly asked to include (1) a description of each vulnerability, (2) the reasoning behind it, and (3) the potential impact if exploited, as shown in Fig. 2. We require the agent to include its reasoning, as previous studies [8, 30, 56] show that prompting LLMs to articulate their reasoning can significantly enhance their performance. Furthermore, including the potential impact of each vulnerability gives the other agents a clearer perspective when they form their answer.

2.2 Code Author Agent (Defense Attorney Role)

The *code author agent* in VULTRIAL acts similarly to a defense attorney in a courtroom scenario. While the prosecutor aims to prove that the defendant is guilty, the defense attorney counters these allegations by advocating for the defendant's innocence. Applying this scenario to vulnerability detection, the *code author agent* seeks to defend the integrity and security of their code, providing arguments and evidence to counter the *security researcher agent's* claims. If the vulnerability is valid, the agent can acknowledge the issue and propose effective mitigation strategies.

The prompt provides the agent with the code snippet under investigation, along with information generated by the *security researcher agent*, which outlines a list of potential vulnerabilities [60]. The agent is then tasked with clearly articulating the following outputs: (1) identification of the vulnerability being discussed, (2) explicit acknowledgment or refutation of the vulnerability, and (3) detailed reasoning justifying the refutation or mitigation methods in case the vulnerability is acknowledged. An example of the *code author agent's* output is also shown in Fig. 2, where it refutes one vulnerability and provides reasoning for it.

Since the *security researcher agent* may highlight multiple potential vulnerabilities, it is crucial for the *code author agent* to clearly specify which vulnerability it addresses in each response. Explanations from the *code author agent* enable other agents (the *moderator* and *review board*) to better understand the justification or rationale behind the agent's position. For the *security researcher agent*, this reasoning would be used for building their rebuttal. This structured approach helps ensure that subsequent agents are equipped to offer well-informed counterarguments, summaries, or final evaluations.

2.3 Moderator Agent (Judge Role)

Just as judges provide guidance to jurors and uphold fairness in a courtroom scenario, within the context of vulnerability detection, this agent is tasked with objectively summarizing the arguments presented by both the *security researcher agent* (prosecutor) and the *code author agent* (defense attorney). It should focus only on the facts, avoiding any bias toward either side. This impartial summary helps the *review board agent* in reaching a decision, while also clarifying critical discussion points for subsequent rounds of argumentation by the *security researcher* and *code author agents*.

In addition to the code snippet, the *moderator* receives the *security researcher's* identified vulnerabilities and the *code author's* counterarguments. The *moderator's* output consists of concise summaries of these two viewpoints, highlighting the key points from each. Fig. 2 shows an example of the *moderator agent* summarizing the arguments from the *security researcher* (e.g., the number of potential vulnerabilities identified, their main reasons, and impacts) as well as from the *code author*, such as which vulnerabilities were acknowledged or refuted with the main reasoning.

2.4 Review Board (Jury Role)

In a traditional courtroom, the jury weighs all evidence and testimony presented during a trial, alongside any guidance from the judge, to determine whether a defendant is guilty or not. Similarly, in vulnerability detection, the *review board agent* is tasked to evaluate the code's security status based on the evidence presented

<pre>TfLiteStatus EvalGatherNd(TfLiteContext* context, const TfLiteTensor* params, const TfLiteTensor* comment_indices, TfLiteTensor* output) { switch (params->type) { case kTfLiteFloat32: return GatherNd<float, IndicesT>(params, indices, output); // ... other types ... default: context->ReportError(context, "Params type '%s' not supported by gather_nd.", TfLiteTypeGetName(params->type)); return kTfLiteError; } }</pre>	
Security Researcher	
vulnerability	: Out-of-Bounds Read
reason	: The function 'EvalGatherNd' does not perform any bounds checking on the 'indices' tensor. [...]
impact	: Exploiting this vulnerability could lead to information disclosure, [...] could be leveraged for further attacks such as Denial of Service (DoS).
vulnerability	: Type Handling
reason	: [...] if the new type is not properly validated [...], it could lead to unexpected behavior or crashes.
impact	: [...] could be exploited by an attacker to cause a DoS
Code Author	
vulnerability	: Out-of-Bounds Read
reason	: The concern regarding out-of-bounds reads is valid. [...] I propose implementing a bounds checking [...].
vulnerability	: Type Handling
reason	: [...] any unsupported type will be caught in the default case, which reports an error. This is a standard practice [...].
Moderator	
researcher summary	: [...] could result in information disclosure [...]
author summary	: The author acknowledges the concern [...].
Review Board	
vulnerability	: Out-of-Bounds Read
severity	: high
reason	: [...] function does not perform bounds checking [...]
decision	: valid
action	: fix immediately
vulnerability	: Type Handling
severity	: medium
reason	: [...] potential for new tensor types w/o proper handling [...]
decision	: partially valid
action	: monitor

Figure 2: Example of vulnerable code (CVE-2021-37687) and analysis results from VULTRIAL. Green text indicates correct arguments, while orange text indicates misleading or exaggerated claims. The code is vulnerable as it does not check for negative indices, allowing an attacker to read arbitrary data by crafting a model with negative values in indices.

by both the *security researcher* and *code author agents*, as well as the *moderator's* summary. Its purpose is to deliver a final verdict on whether the potential vulnerabilities identified by the *security researcher* are indeed valid. This agent receives (1) the summary from the *moderator agent*; (2) the list of potential vulnerabilities from the *security researcher agent*; (3) the *code author agent's* counterarguments; (4) the original code snippet.

The *review board agent* is tasked with generating several outputs. First, it must extract the names of potential vulnerabilities from the list provided by the *security researcher* and proceed with a decision for each one. Second, the agent must determine whether each identified vulnerability is "valid," "invalid," or "partially valid." This process is similar to a jury deciding whether a defendant is guilty or not guilty. Third, the agent must assess the severity of the vulnerability, similar to how a jury determines the extent of damages in a trial. However, instead of assigning an open-ended value, the *review board* needs to categorize severity as "low," "medium," or "high." If the vulnerability is deemed invalid, the severity is marked as "none." Fourth, the agent must provide a recommended action based on the decision and severity level. For example, if a vulnerability is deemed valid and has a high severity, the recommended

action should be “fix immediately.” Finally, the agent must generate an explanation for its decisions. Unlike a real jury, we require the *review board agent* to provide reasoning, as it improves LLMs’ performance [8, 30, 56] and helps users understand the decision.

We use a threshold rather than a simple binary “vulnerable or not,” to give users flexibility in balancing risk and false positives. If a user prefers to catch even the smallest chance of vulnerability, they can opt to flag any partially valid vulnerabilities. Conversely, if a user wants fewer false positives, they can raise the threshold.

We flagged a function as vulnerable if at least one vulnerability raised by the *security researcher agent* is deemed valid at high severity with recommended action fix immediately by the *review board agent*. This threshold ensures that only the critical and well-substantiated vulnerabilities contribute to the classification, reducing false positives from lower-risk or ambiguous issues. We aim to reduce noise by excluding vulnerabilities marked as partially valid or those with lower severity levels. Fig. 2 also includes an example of the review board agent’s output. It classifies the first vulnerability (Out-of-Bounds Read) as “valid” with “high” severity. Meanwhile, the second vulnerability (Type Handling) is marked as “partially valid” with “medium” severity. As at least one confirmed vulnerability is present, the final verdict for this code is vulnerable.

2.5 Agent Interaction

In VULTRIAL, four agents interact sequentially: the *security researcher agent*, the *code author agent*, the *moderator agent*, and finally the *review board agent*. During the first round, the *security researcher agent* (1) has access only to the original code snippet. In subsequent rounds, it also receives information from the *code author* and *moderator*. Similarly, the *code author agent* (2) initially sees only the *security researcher agent*’s findings, but in the next rounds, it can also incorporate the *moderator*’s (3) output. This setup is similar to a courtroom process, which can allow for multiple rounds of arguments and counterarguments between the prosecutor, defense attorney, and judge. The *review board agent* (4), mirroring a jury role, provides its verdict only after all discussions have concluded. It does not participate in the debate itself but bases its final decision on the evidence and summaries presented by the other three agents.

Allowing multiple discussion rounds may enhance the iterative refinement of analyses and arguments. It also provides both the *security researcher* and *code author agents* with more opportunities for rebuttal. In this study, we initially limit the number of discussion rounds to one (Section 4.1) and then assess how changing the number of rounds affects VULTRIAL’s performance (Section 4.3).

3 Experimental Setup

3.1 Dataset

We utilized the PrimeVul dataset, recently proposed by Ding et al. [12], to evaluate vulnerability detection approaches. Specifically, we used the PrimeVul pair dataset, which presents a challenging benchmark, as the vulnerable and benign code functions share at least 80% of their string content. The pair dataset includes 3,789 training pairs, 480 validation pairs, and 435 test pairs, with no data leakage across splits. In contrast, the full PrimeVul dataset contains 175,797 training functions (4,862 vulnerable and 170,935 benign) and 23,948 validation functions (593 vulnerable and 23,355 benign).

3.2 Research Questions

This study aims to answer the following research questions (RQs):

RQ1: How does the performance of VULTRIAL compare to that of single-agent and multi-agent baselines on PrimeVul? Prior work [12] demonstrated the poor performance of GPT-3.5 and GPT-4 for vulnerability detection on pairwise vulnerable and benign functions in PrimeVul. These findings highlight the limitations of single-agent prompting strategies for this challenging setting. In this RQ, we investigate whether VULTRIAL, a multi-agent LLM-based framework (Section 2), can outperform both single-agent methods [12] and existing multi-agent approaches such as GPTLens [26] on the PrimeVul pair.

RQ2: How does instruction tuning on role-specific agents influence the performance of automated vulnerability detection? VULTRIAL consists of four agents, each with a specific role. Instruction tuning can be applied to each agent to strengthen its performance in its designated task. In this RQ, we investigate whether this role-specific instruction tuning can enhance the overall performance of VULTRIAL.

RQ3: Does allowing multiple rounds of debate among role-specific agents in VULTRIAL lead to improvement compared to a single-pass analysis? In previous RQs, we focused on conducting a single round of discussions among agents to reach a final verdict. However, allowing multiple rounds of discussion could give agents the opportunity to refine their arguments to reach a more accurate decision. In this RQ, we examine whether enabling multiple rounds of discussion can enhance VULTRIAL’s performance.

RQ4: How does each agent contribute to the overall performance of VULTRIAL? VULTRIAL features four distinct roles, *security researcher*, *code author*, *moderator*, and *review board*, which is designed to follow a courtroom-style scenario. While the results from previous RQs highlight VULTRIAL’s effectiveness, it is not known whether each agent is essential to the final prediction and how it impacts the performance. In this RQ, we do the ablation study that omits one agent at a time to verify each agent’s contribution.

3.3 Baseline and Models

Baselines. We employed both DL-based and LLM-based vulnerability detection techniques as single-agent baselines. For the DL-based methods, we used several state-of-the-art models widely adopted in prior vulnerability detection studies [7, 12, 64, 68]:

- CodeBERT [16]: A bimodal pre-trained transformer model trained on source code and natural language.
- CodeT5 [55]: A transformer model that learns to understand and generate code across multiple programming languages.
- UniXCoder [21]: A cross-modal pre-trained model that jointly learns representations of code, ASTs, and execution traces.
- LineVul [18]: A line-level transformer-based vulnerability prediction model detecting vulnerable lines through fine-grained context representations.

For CodeBERT, CodeT5, and UniXCoder, we performed two separate fine-tuning: (1) using the full PrimeVul training and validation sets, and (2) using only the pair-wise subsets from PrimeVul. For LineVul, we first evaluated the model provided in the previous study [18]. In addition, we further fine-tuned LineVul using the pair-wise subset to ensure a consistent comparison with the other

DL models. For the LLM-based approach, we utilized the chain of thought (CoT) [56] prompting technique, which achieved the highest results in the PrimeVul [12]. CoT prompting encourages LLMs to reason through problems step by step to arrive at an answer.

For the multi-agent baseline, we utilized an approach from a recent study, GPTLens [26]. GPTLens operates with two types of agents: *auditor agent*, which identifies potential vulnerabilities, and a *critic agent*, which evaluates and scores the identified vulnerabilities. The final decision is based on the critic's score; if the score is less than 5, it is classified as benign; otherwise, it is considered vulnerable. There are several key differences between GPTLens and VULTRIAL. VULTRIAL follows a courtroom-inspired setting with four role-specific agents, whereas GPTLens relies on a two-agent structure. Unlike GPTLens, where the *critic agent* determines the final decision, VULTRIAL used a different agent (i.e., *review board agent*) for this task, designed to be neutral toward the *security researcher* and *code author agents*. Recent studies [19, 27] suggest that employing domain-specific multi-agent systems that reflect a more realistic scenario leads to better decision making, further supporting our approach in VULTRIAL.

Models. We used well-established OpenAI models known for their strong performance, as confirmed by prior research [26, 59, 69].

GPT-3.5 [38]: GPT-3.5 is one of the first widely popular OpenAI LLM models. Following PrimeVul [12], we also evaluate the performance of this model in our study. In our evaluation, we utilized *gpt-3.5-turbo-0125*, the latest iteration of the GPT 3.5 model with a context window of 16k tokens.

GPT-4o [41]: We chose GPT-4o, one of OpenAI's flagship models, over GPT-4 [39] (used in PrimeVul's evaluation) because OpenAI indicates that GPT-4o offers performance comparable to GPT-4 while significantly reducing computational costs [41]. Specifically, we used *gpt-4o-2024-08-06* allows up to 128k tokens in its context window and 16k output tokens.

3.4 Evaluation Metrics

Following the approach in PrimeVul [12], we adopt a pair-wise evaluation strategy. It is designed to assess the model's prediction on pairs of vulnerable and benign code, rather than the single codes. This method highlights the model's capacity to differentiate subtle variations that make one function vulnerable and the other benign. We report five metrics. Pair-wise Correct Prediction (P-C) records pairs where both functions are classified correctly. Pair-wise Vulnerable Prediction (P-V) records pairs where both functions are incorrectly labeled as vulnerable, while Pair-wise Benign Prediction (P-B) records pairs labeled incorrectly as benign. Pair-wise Reversed Prediction (P-R) captures cases where the model assigns opposite labels to the pair, predicting "benign" for the "vulnerable" code and "vulnerable" for the "benign" one. Pair-wise Error (E) reflects pairs in which one function is misclassified. Among these metrics, P-C is our primary evaluation metric. A high P-C indicates accurate vulnerability detection that avoids false positives and negatives, making it a more robust indicator of overall performance.

To provide more comprehensive discussion in VULTRIAL and baselines comparison, we also report standard classification metrics: Precision (P), Recall (R), and False Positive Rate (FPR), following the formula used in the previous study [12]. These traditional metrics

provide additional insights into the model's general classification behavior, particularly in individual predictions.

3.5 Methods

3.5.1 Answering RQ1: We compare VULTRIAL with both single-agent (CodeBERT [16], CodeT5 [55], UniXCoder [21], LineVul [18], and Ding et al.'s CoT[12]) and multi-agent (GPTLens) baselines. For the LLM-based techniques, we utilize two different LLMs, i.e., GPT-3.5 and GPT-4o (see Section 3.3). We evaluate each approach using several metrics (P-C, P-V, P-B, P-R, Error, P, R, and FPR).

3.5.2 Answering RQ2: For this RQ, we conducted instruction tuning using samples drawn from GPT-4o's validation set. After running GPT-4o on the set, we collected its correct outputs to build an instruction-tuning dataset for a GPT-4o model. We focus on GPT-4o because, as shown in RQ1, VULTRIAL with GPT-4o outperforms both single-agent and multi-agent baselines and achieves the best overall results. Our goal is to determine whether instruction tuning can further enhance its performance. We chose GPT-4o for generating the instruction data to ensure consistency in style and reasoning patterns. By using the same models for generating data, we reduce the risk of mismatches that can occur when instruction tuning with outputs from a completely different model.

For instruction tuning data, we use 50 code pairs (vulnerable and benign) with the fewest tokens in the validation set to help manage the tuning cost. We chose 50 samples based on preliminary experiments comparing different dataset sizes (25, 50, and 75 pairs), as 50 resulted in significant performance gains over 25 samples, while achieving comparable results to using 75 samples [60]. In addition, OpenAI has highlighted that GPT-4o can achieve strong results with "as few as a few dozen examples" [40]. When creating our instruction tuning dataset, we initially included the true label in the prompt, but then removed direct label disclosures in the final dataset. For example, for the code author agent [60], we included vulnerability-related information such as CVE ID and label. Similarly, for the security researcher agent, we added the CVE description and label for both vulnerable and benign code, prompting the agent to connect code to relevant vulnerabilities. For the review board agent, the prompt included ground-truth labels but also emphasized that the agent should not reference them explicitly in its reasoning. In contrast, the moderator agent's prompt excluded any ground-truth vulnerability information and was cleaned to avoid indirect label leakage from chat history. As a result, each prompt retained its original role-specific structure and was paired with the correct assistant output.

For this instruction tuning data, we ensure that the agent's predicted label ("vulnerable" or "benign") matches the ground-truth label. Additionally, we manually verify the quality of the agent's explanation, confirming that it correctly identifies the presence or absence of a vulnerability. An example of the vulnerable instance is shown in the online appendix [60]; this vulnerability is classified as CWE-416: Use After Free. The security researcher agent correctly identifies the issue within the CleanWriters function, where the sequence of operations may lead to undefined behavior if `gf_isom_box_del()` uses the writer object after it has been freed. The explanation aligns well with the root cause described in CVE-2020-35980. For the corresponding benign variant [60], the

code author agent correctly refutes the presence of a vulnerability by noting that `gf_list_del_item()` only removes the items from the list without deallocating their memory. The justification provided by the *review board agent* also matches the *code author's agent* clarification and supports the benign ground-truth label.

We evaluated eight scenarios: (1) no instruction tuning; (2-5) tuning only the *security researcher*, *code author*, *moderator*, or *review board agent*; (6) tuning *security researcher* and *code author*; (7) tuning *security researcher*, *code author*, and *moderator*; and (8) tuning all agents. To systematically assess the impact of instruction tuning, we first applied it to individual agents to understand their specific contributions. Then, we incrementally introduced instruction tuning in the order of agent interaction, allowing us to analyze how tuning at different stages influences the overall performance of VULTRIAL. This stepwise approach provides insights into the cumulative effects of instruction tuning across multiple agents.

3.5.3 Answering RQ3: We experimented with three values of k (i.e., the number of interaction rounds): 1, 2, and 3 in VULTRIAL. We selected these values to systematically investigate the impact of multiple interaction rounds while maintaining computational feasibility. For the base model, we used GPT-4o with an instruction-tuned agent, as it yielded the strongest overall performance.

3.5.4 Answering RQ4: In this RQ, we conduct an ablation study by systematically removing each agent to assess its necessity. Since the *security researcher agent* is fundamental for identifying potential vulnerabilities, we retain it in all scenarios. Additionally, since the *moderator* primarily provides summaries rather than making explicit decisions on vulnerabilities, we do not include a scenario where the *moderator* is responsible for delivering the final verdict.

We evaluated four scenarios: (1) the full VULTRIAL (baseline) includes all four agents; (2) removing the *moderator*; (3) removing the *code author*; (4) removing two agents simultaneously, using either (a) only the *security researcher* and *review board* or (b) only the *security researcher* and *code author*. For scenario 4(b), as the final decision is made by the *code author agent*, we classify the code as vulnerable if it mitigates at least one of the potential vulnerabilities.

By comparing performance across these scenarios, we can identify which agents play the most critical roles. For example, does a *moderator* assist the *review board* in reaching a final decision, and how essential is a different agent for delivering verdicts? This ablation would highlight the necessity of each agent in VULTRIAL.

3.6 Implementation Details

We built VULTRIAL on top of AgentVerse [6], a recently developed framework that simplifies the deployment of multiple LLM-based agents. For single-agent baselines, we used the prompt in PrimeVul [12], while for GPTLens [26], we employed the provided replication package [25]. Because the original GPTLens study focused on smart contracts, we adapted its prompt by replacing references to “smart contract” with “code” to generalize the use case. We used the best settings reported in their study, including two auditor agents and a top- k value of 3. We counted the results as vulnerable if they got a final score of at least 5 (out of 9). All GPT-based models were accessed via the OpenAI API, which was also used to generate

instruction tuning data. For the fine-tuning itself, we used OpenAI's built-in dashboard feature for custom GPT-3.5 and GPT-4o instruction tuning [40]. To make the results more deterministic, we set the temperature to 0 for both our single-agent baselines and VULTRIAL. For GPTLens [26], we followed the same setting used in its evaluation (0.7 for the auditor agent and 0 for the critic agent).

4 Results

4.1 RQ1: VULTRIAL vs. Baselines

Table 1 presents the performance of VULTRIAL against both single-agent (CodeBERT [16], CodeT5 [55], UniXCoder [21], LineVul [18], and Ding et al.'s CoT prompts [12]) and multi-agent (GPTLens [26]) baselines. In addition to the pair-wise metrics [12] (P-C, P-V, P-B, P-R, Error), we also report traditional metrics (P, R, FPR).

Deep learning-based Method. Our results indicate that DL models trained on the balanced pair dataset fail to effectively differentiate between vulnerable and benign functions. While these models achieve nearly perfect recall ($R=0.96-1.00$), it comes at the cost of maximum false positive rates ($FPR=0.99-1.00$). This leads to zero or near-zero correct predictions for both functions in a pair ($P-C=0-6$). When trained on the full dataset (imbalanced), DL models slightly improve, reducing FPR to approximately 0.10-0.14; however, they still struggle significantly, with very few correct pair predictions ($P-C \leq 7$) and low recall ($R \leq 0.13$). Most functions are incorrectly labeled benign ($P-B=336-403$), indicating that the models cannot pick up subtle vulnerability patterns amid class imbalance, so they miss most of the real vulnerability.

These results highlight the limitations of traditional DL models in capturing subtle vulnerability patterns at the function-pair level. While DL-based models are more lightweight and computationally efficient, our results demonstrate that this efficiency comes at a cost in actionable performance. For example, compared to the highest performing DL baseline, LineVul, achieves only 7 for P-C, with a modest FPR of 0.21 and a recall of 0.22. VULTRIAL (using GPT-3.5) achieves a P-C of 68 (nearly 10× higher) while maintaining a comparable FPR and recall of 0.23 and 0.25. With GPT-4o, VULTRIAL pushes the P-C even further to 81 (12× higher than LineVul), with recall reaching 0.60. Although this comes at the cost of a higher FPR (0.52), the trade-off results in vastly more correct pairwise classifications and a significantly higher yield of true vulnerabilities.

LLM-based Method (GPT-3.5). The single-agent CoT prompt obtains modest accuracy ($P-C=18$, $FPR=0.08$) but misses most vulnerabilities ($R=0.08$). Meanwhile, GPTLens boosts recall to 0.94; however, it comes with a high FPR of 0.95 with a small number of correct pairs (20). VULTRIAL obtains 68 correct pairs, which is 3.8× the CoT baseline and 3.4× GPTLens, while keeping FPR to 0.23.

GPTLens is configured to output the top-3 potential vulnerabilities per function, following the best setup from the original study [26]. Since the *auditor agent* runs twice, this results in up to six potential vulnerabilities per function. Indeed, most *auditor* outputs list six vulnerabilities, with an average of 5.6 and a median of 6. Meanwhile, VULTRIAL produces an average of 1.74 vulnerabilities per function, with a median of 2. In other words, VULTRIAL produces fewer vulnerabilities compared to GPTLens, but it has more precise vulnerability detection, shown by a higher P-C.

Table 1: Results of various vulnerability detection methods on PrimeVul paired functions. An upward-pointing arrow (↑) indicates that a higher score is better, while a downward-pointing arrow (↓) indicates that a lower score is better.

Model	Agents	Method	P-C ↑	P-V ↓	P-B ↓	P-R ↓	Error ↓	P ↑	R ↑	FPR ↓
CodeBERT [16]	Single-Agent	Fine-tuned full [†] [12]	5	37	386	7	430	0.49	0.10	0.10
		Fine-tuned pair [‡]	0	435	0	0	435	0.50	1.00	1.00
CodeT5 [55]	Single-Agent	Fine-tuned full [†] [12]	0	57	373	5	435	0.48	0.13	0.14
		Fine-tuned pair [‡]	0	434	1	0	435	0.50	0.99	0.99
UniXCoder [21]	Single-Agent	Fine-tuned full [†] [12]	4	26	403	2	431	0.52	0.07	0.06
		Fine-tuned pair [‡]	6	412	17	0	429	0.50	0.96	0.95
LineVul [18]	Single-Agent	Fine-tuned [§] [18]	7	89	336	3	428	0.51	0.22	0.21
		Fine-tuned pair [‡]	0	434	1	0	435	0.50	0.10	1.00
GPT-3.5	Single-Agent	Ding et al.'s CoT [12]	18	16	381	20	417	0.49	0.08	0.08
	Multi-Agent	GPTLens [26]	20	388	3	24	415	0.50	0.94	0.95
		VULTRIAL	68	40	265	62	367	0.51	0.25	0.23
GPT-4o	Single-Agent	Ding et al.'s CoT [12]	40	43	323	29	395	0.53	0.19	0.17
	Multi-Agent	GPTLens [26]	44	241	122	28	391	0.51	0.65	0.62
		VULTRIAL	81	179	125	50	354	0.53	0.60	0.52

[†]Fine-tuned on full PrimeVul train and val sets (4,862 vuln / 170,935 benign). [‡]Fine-tuned on paired PrimeVul train and val sets (3,789 vuln / 3,789 benign).

[§]Fine-tuned model from prior work [18].

LLM-based Method: GPT-4o. VULTRIAL achieves the highest number of pair-wise correct predictions (P-C=81), outperforming both the CoT baseline (P-C=40) and GPTLens (P-C=44). This is 2.03× the performance of the single-agent baseline and 1.84× that of GPTLens. These gains highlight VULTRIAL's effectiveness at correctly identifying both the vulnerable and benign functions within a pair that have subtle differences.

In terms of traditional classification metrics, VULTRIAL achieves a precision of 0.53, on par with the CoT baseline (0.53) and slightly above GPTLens (0.51). Its recall is 0.60, below GPTLens (0.65), but significantly higher than CoT (0.19). VULTRIAL FPR is 0.52, lower than GPTLens (0.62) but higher than CoT (0.17). This trade-off illustrates VULTRIAL's balanced behavior; it identifies more true vulnerabilities than CoT while maintaining a lower FPR than GPTLens. In terms of P-B, which measures how often both functions in the pair were incorrectly classified as benign, VULTRIAL (P-B=125) is significantly lower than the CoT baseline (P-B=323) and comparable to GPTLens (P-B=122). Despite these improvements, VULTRIAL does not outperform the alternatives on every metric. GPTLens still achieves the highest recall, making it preferable in scenarios where catching as many vulnerabilities as possible is critical.

In the GPTLens experiment using GPT-4o, the first agent highlights a similar number of potential vulnerabilities as it did under GPT-3.5, with a median of 6 and a mean of 5.76. Meanwhile, VULTRIAL with GPT-4o reports fewer vulnerabilities (median 4, mean 4.11) compared to GPTLens. Despite highlighting fewer vulnerabilities, VULTRIAL achieved higher P-C and lower P-B scores than GPTLens. This suggests that the additional agents in VULTRIAL, can effectively distinguish between justified and unjustified vulnerability claims, leading to more accurate final predictions.

Our qualitative analysis further illustrates how VULTRIAL's structured, role-specific dialogue improves both fidelity and precision of vulnerability detection. Each agent in VULTRIAL has a specific role. The *security researcher agent* analyzes the function to identify vulnerabilities. For example, in Fig. 2, the *security researcher* highlights

two vulnerabilities: (1) an Out-of-Bounds Read due to unvalidated negative indices (CWE-125), and (2) a Type Handling issue. The first is a valid case, while the second reflects an overcautious concern since the existing code already handles unsupported types safely. After reviewing these findings, the *code author agent* agrees with the Out-of-Bounds Read assessment but refutes the Type Handling concern. The *moderator agent* then summarizes key points from both agents, which aim to help the *review board agent* in making the decision. Based on this information, the *review board agent* concludes that the Out-of-Bounds Read is a genuine vulnerability. Meanwhile, for the Type Handling issue, the *review board* agrees with the *code author's* argument that the existing implementation is safe. However, it's concerned about potential future changes that could introduce vulnerabilities, as highlighted by the *security researcher agent*. As a result, this issue is classified as partially valid, which is considered benign since it falls below the threshold. This example illustrates how the agents collectively separate confirmed vulnerabilities from exaggerated ones, preventing over-reporting and guiding appropriate remediation.

Another example of running VULTRIAL on vulnerable code can be found in Appendix H [60]. The *security researcher agent* flags the lack of strict checks; the *code author* proposes callbacks to mitigate the issue, and the *review board* ultimately rules it a high-severity, valid vulnerability requiring immediate fix. Here, both the *security researcher* and *code author* align, reinforcing the confidence of the final judgment. This demonstrates VULTRIAL's capacity to not only support adversarial debate but also consensus when warranted.

Conversely, in the example of the benign function that can be found in Appendix I [60], the *security researcher agent* initially raises a concern about certificate validation. The *code author agent* points out that X.509 verification is already used, which the review board agent independently confirms. The issue is ruled invalid. This case highlights VULTRIAL's strength in filtering false positives as the security researcher agent's overstatement is corrected by technical context from the code author's agent.

Table 2: Results of VULTRIAL using GPT-4o with instruction-tuned agents: SR (*security researcher*), CA (*code author*), M (*moderator*), and RB (*review board*).

Instruction-tuned	P-C ↑	P-V ↓	P-B ↓	P-R ↓
None	81	179	125	50
<i>Security researcher</i> (SR)	54	153	187	41
<i>Code author</i> (CA)	52	39	314	30
<i>Moderator</i> (M)	96	154	134	51
<i>Review board</i> (RB)	72	133	183	47
SR + CA	34	40	331	30
SR + CA + M	32	37	338	28
SR + CA + M + RB	45	46	305	39

In another example of benign function (Appendix J [60]), the *security researcher agent* flags potential leakage via verbose logging. The *code author* proposes mitigations: increasing verbosity thresholds, securing log storage, and reviewing logged content. The *moderator* captures this exchange. After reviewing the evidence, the *review board* classifies the issue as partially valid with low severity (which is below the vulnerability threshold), opting to monitor rather than patch, thereby filtering out an overcautious report.

Our qualitative analysis shows that the four agents contribute different kinds of knowledge: pattern detection (*security researcher*), code-context refutation or mitigation (*code author*), factual summarisation (*moderator*), and balanced final judgment (*review board*). This minimizes errors on both ends: they surface genuine threats while filtering out already-handled or overstated concerns, reducing false negatives and false positives.

Answer to RQ1: VULTRIAL achieves the highest number of correct function pair predictions. It more than doubles the number of fully-correct function pairs relative to baselines, while avoiding the extreme false-positive rates and the low recall.

4.2 RQ2: Instruction Tuning on Agents

From the RQ1’s results, we observed that VULTRIAL using GPT-4o achieves the best performance. In this RQ, we investigate whether instruction tuning on role-specific agents can further improve VULTRIAL’s overall performance. The results are shown in Table 2.

GPT-4o Instruction Tuning. All the single instruction-tuned variants of GPT-4o in VULTRIAL, covering the *security researcher agent*, *code author agent*, *moderator agent*, and *review board agent*, outperformed both the single-agent and multi-agent baselines. Among these, tuning the *moderator agent* delivered the most substantial performance gain. Specifically, the moderator-tuned version increases the performance of the non-tuned version in VULTRIAL by 15 P-C scores. Compared to the single-agent baseline, this instruction-tuned version of VULTRIAL increases P-C score by 56. While for the multi-agent baseline, the improvement of the instruction-tuned version of VULTRIAL increases P-C score by 52.

Effect of Agent-Specific Tuning. Upon further analysis of the instruction-tuned *moderator agent*, we observed that its summaries are more concise compared to the non-tuned version. Specifically,

Table 3: Results of VULTRIAL using GPT-4o across different numbers of discussion rounds.

Model	#Round	P-C ↑	P-V ↓	P-B ↓	P-R ↓
GPT-4o [‡]	1	96	154	134	51
	2	87	173	122	53
	3	81	194	110	50

[‡] Instruction tuning for *moderator agent*.

the tuned version of the moderator agent provides a shorter summary than the non-tuned version, reducing the summary length for *security researchers* by 16.03% and for *code authors* by 16.58%. In cases where the tuned VULTRIAL correctly classified code as benign but the non-tuned version did not, the *security researcher’s* summary was 21.31% shorter in the tuned version. Meanwhile, when the tuned VULTRIAL correctly classified code as vulnerable but the non-tuned version did not, the *code author’s* summary was 19.03% shorter. These observations suggest that the instruction-tuned *moderator agent* can effectively distill key points from both the *security researcher* and *code author*, which help the *review board agent* to make more accurate final verdicts.

We next examined the impact of instruction tuning on the *security researcher* and *code author* agents. Although both configurations still surpassed single-agent and prior multi-agent baselines, their performance fell below that of the non-tuned VULTRIAL. After tuning, we found that the *security researcher agent* proposed fewer potential vulnerabilities, with an average of 2.58 per code instance compared to 4.11. To generate the tuning data, GPT-4o is provided with ground truth information, such as CVE details. Since it already knows the ground truth vulnerabilities, the number of potentially vulnerable instances identified by *security researcher agent* is smaller. On average, *security researcher* identify 1.25 vulnerabilities in the training data, which is lower compared to the non-tuned version. This low number of vulnerabilities in the tuning data leads to a lower number of vulnerabilities identified in the tuned version of *security researcher agent*.

Similarly, instruction tuning the *code author agent* altered its mitigation-to-refutation ratio. While the mitigation to refutation ratio of the non-tuned version was 2:1, the tuned version exhibited a 1:4 ratio. In other words, the tuned *code author* refuted vulnerabilities more frequently. From its tuning data, it learned to doubt potential vulnerabilities highlighted by *security researcher*. While this can restrict over-diagnosing vulnerabilities, it risks undervaluing genuine security concerns, as highlighted by an increase in P-B. Despite outperforming the single-agent baseline, this shows the delicate balance required for instruction tuning. While instruction tuning can sharpen an individual agent’s skills, it may introduce biases or extreme behaviors that undermine collective performance.

Combining Agent-Specific Tuning. We also explored configurations with multiple tuned agents, such as the *security researcher* and *code author*, are used together in VULTRIAL. These combinations did not surpass the performance achieved by tuning only the *moderator agent*. For instance, when all agents were instruction-tuned, the average number of identified vulnerabilities dropped from 4.11 to 2.60, and the refutation-to-mitigation ratio shifted to 1:4. This imbalance corresponded to a higher P-B (331). These lower

results may stem from the instruction tuning setup, where the review board is trained on histories that include inputs from other agents. In the tuning data, the review board often learns to “follow” the code author’s position (especially strong arguments), which is aligned with the ground truth. Once the code author is instruction-tuned and becomes more assertive in refuting vulnerabilities, the review board (also fine-tuned) tends to side with the author. Future work should decouple agent training, for example, including cases where the review board correctly identifies a vulnerability despite strong refutations, to reduce over-alignment and improve decisions.

Answer to RQ2: Instruction tuning for *moderator agent* using 50 paired samples yields the most significant improvements. It improves P-C by 15, 56, and 52 compared to the non-tuned version, single-agent, and multi-agent baselines, respectively.

4.3 RQ3: Effect of Multi Rounds Discussion

In the previous RQs, VULTRIAL used a single round of interaction among the agents. In this RQ, we investigate whether additional discussion rounds enhance performance. Table 3 shows the results of these multiple rounds experiments, showing that the best result is achieved using a single round interaction. For GPT-4o, increasing the number of rounds actually led to a decrease in overall performance. Specifically, P-V increases with more rounds. From further analysis, we found that the *security researcher agent* suggested fewer potential vulnerabilities in later rounds, with an average of 4.22 per code in Round 1, decreasing to 3.55 in Round 2 and 3.43 in Round 3. This decline is also driven by the *code author agent*’s refutations. In 30.88% of the total predictions, the *security researcher* removed a previously identified vulnerability after the code author disputed it. Meanwhile, the *code author agent* became progressively more inclined to acknowledge vulnerabilities, with its mitigation-to-refutation ratio shifting from 1.73:1 in Round 1, to 2.5:1 in Round 2, and finally 3:1 in Round 3. Although the *review board agent* did not blindly accept arguments from either side (for instance, during the three-round setup, in 100 out of 868 code instances, *review board agent* overruled the *code author*’s refutation and decided the vulnerability to be valid), the overall imbalance introduced by these interactions still raised P-V and lower P-C rates.

Answer to RQ3: Using GPT-4o in VULTRIAL outperforms both single-agent and multi-agent baselines across different rounds of agent interaction (i.e., 1, 2, and 3). However, performance declines when extending beyond a single discussion round.

4.4 RQ4: Ablation Study

In this RQ, we investigate the necessity of each agent in VULTRIAL. Table 4 presents the ablation results under different agent-removal scenarios. Overall, all four agents are essential for achieving the best performance. Removing the *moderator agent* reduces P-C from 96 to 32, highlighting its critical role in summarizing arguments to help *review board agent* in making the decision. This aligns with the results of RQ2 (see Section 4.2), where instruction tuning the *moderator* yielded the largest performance gain. Removing both the *moderator* and *review board agent* causes P-C to further drop to 9, while P-V (pairs both labeled vulnerable) increases to 407.

Table 4: Results of ablation study on agent in VULTRIAL using GPT-4o with moderator agent tuned.

Agents	P-C ↑	P-V ↓	P-B ↓	P-R ↓
Base	96	154	134	51
No <i>moderator</i>	32	37	338	28
No <i>code author</i>	63	277	52	43
No <i>moderator</i> + <i>code author</i>	63	269	63	40
No <i>moderator</i> + <i>review boards</i>	9	407	5	14

Highlighting that most code is labeled vulnerable by the *code author*. This imbalance occurs because, without an additional agent *review board*, the final judgment skews toward labeling codes as vulnerable based on partial agreements from the *code author agent*. Thus, the review board is essential for ensuring balanced final verdicts.

Removing the *code author agent* reduces P-C to 63 and increases P-V to 277. Without the *code author*’s counterarguments, the *review board agent* has a limited perspective and often accepts vulnerabilities identified by the *security researcher agent*. Removing both the *moderator* and *code author* produces similar results (P-C=63, P-V=269). This underscores the crucial role of the *code author* in ensuring balanced judgments and showing that the *moderator agent* becomes redundant without opposing arguments to mediate.

Answer to RQ4: Our findings confirm that every agent in VULTRIAL is necessary to achieve the best performance.

5 Discussion

Cost vs. Performance Trade-offs. By assigning specialized roles to agents, mirroring real-world scenarios, VULTRIAL can facilitate deeper discussions by incorporating diverse rationales from different parties. However, this multi-agent approach inherently incurs additional costs due to the increased number of input and output tokens. Interestingly, while GPT-4o is generally more expensive and performs better than GPT-3.5 as a single-agent, using VULTRIAL with GPT-3.5 outperforms GPT-4o’s single-agent P-C by 28. This approach also reduces costs, requiring only \$7.46 compared to GPT-4o’s single-agent cost of \$8.10 for the whole testing data. Conversely, applying VULTRIAL to more expensive models, such as GPT-4o, significantly increases costs (approximately \$23.9=32×8.1). Therefore, careful consideration is important when selecting models to optimize both cost efficiency and detection performance.

Enhancing Role-Specific Agents. Tuning the *moderator agent* in VULTRIAL using GPT-4o improves the baselines and non-tuned version of VULTRIAL. These results highlight the potential for improving performance by tuning role-specific agents. Our study used only a sample of 50 pairs for instruction tuning, keeping the cost at approximately \$17 for each agent. This is significantly lower than using the instruction tuning setting done in PrimeVul [12], which would cost more than \$500 for GPT-4o (>15,000 samples).

Analysis on Instruction Tuning Data. Comparing the 50 fine-tuning pair samples from the validation dataset with the 96 successful VULTRIAL pairs, we found that 39 projects from the successful cases do not appear in the fine-tuning set. Only 8 projects are shared. Of these, 3 projects contain only commits that occurred after the

Table 5: Results of running VULTRIAL in the wild.

Project	Funcs analyzed	Alarms raised	False positives	True positives
Superset [3]	47	0	0	0
DataHub [10]	160	2	0	2
InvenTree [29]	11	1	0	1

corresponding training commits. In total, only 11 successful pairs involve commits made earlier than the latest commit used in training. The fine-tuning data includes 24 unique CWEs, 20 of which also appear in the successful set. 12 CWEs in the successful set are not covered by the fine-tuning data. These results show minimal overlap in terms of projects, commit timing, and CWE types, suggesting that the model’s performance is not due to memorization.

VULTRIAL in the Wild. To assess VULTRIAL in real-world settings, we conducted a pilot experiment on selected open-source projects. Specifically, we targeted well-maintained repositories with active security practices. To identify such projects, we filtered GitHub repositories that include a SECURITY.md file and have over 1,000 stars. We chose the projects that have a SECURITY.md file, as it typically indicates that maintainers are attentive to security concerns and have set up a process for reporting vulnerabilities.

This selection process yielded 196 repositories. From this set, we randomly chose three projects for analysis. For each project, we analyzed recent commits, extracted modified functions, and applied VULTRIAL with its default rule set, no project-specific tuning. The results of this experiment are summarized in Table 5. We manually inspected the flagged vulnerabilities, attempted exploit generation, and reported confirmed issues to the respective maintainers if the vulnerable code was used in the project (i.e., not dead code).

In InvenTree, we discovered a denial-of-service vulnerability in the built-in `label_sheet` plugin. An unbounded `skip` parameter allows attackers to allocate excessively large Python lists and exhaust server memory. This issue affects any authenticated user who can request labels and has been assigned CVE-2025-49000. In DataHub, VULTRIAL identified two functions with resource-exhaustion risks (CWE-400). One function is still actively used, and one is deprecated but remains in the codebase. We disclosed the first issues following SECURITY.md instructions, and the DataHub developers confirmed the findings. These results show that VULTRIAL can uncover previously unknown vulnerabilities in the wild. To reduce maintainer burden, we adopted a conservative reporting strategy, reporting only issues we could confidently confirm as true vulnerabilities. We have made the results, vulnerability reports, and exploits available [61].

Generalizability of VULTRIAL. In our main experiments, VULTRIAL, using OpenAI’s models, outperforms the baseline methods. Next, to investigate its generalizability, we further evaluated it with a different model family, specifically, LLaMA-3.1-8B [15]. Table 6 shows the results, indicating the consistent performance of VULTRIAL even with a different model family. CoT struggles under LLaMA-3.1-8B, with only 4 correct predictions (P-C) and a recall of 0.02, largely due to a high number of benign misclassifications (P-B = 425). VULTRIAL improves recall to 0.48 and raises P-C to 89, although it has an increased false-positive rate (FPR = 0.45).

Table 6: Results of VULTRIAL, single-agent, and multi-agent baselines with LLaMA-3.1-8B as the base model.

Method	P-C↑	P-V↓	P-B↓	P-R↓	E↓	P↑	R↑	FPR↓
CoT[12]	4	3	425	3	431	0.54	0.02	0.01
GPTLens[25]	80	158	103	94	355	0.49	0.55	0.58
VULTRIAL	89	120	150	76	346	0.52	0.48	0.45

This suggests that VULTRIAL favors broader vulnerability coverage, potentially at the cost of mislabeling some benign pairs.

When compared with GPTLens, VULTRIAL presents a different trade-off. GPTLens achieves higher recall (0.55) and fewer benign misclassifications (P-B = 103), but it also produces more P-V (158) and a higher FPR (0.58). VULTRIAL, in contrast, has more correct predictions overall (P-C = 89 vs. 80) and higher precision (0.52 vs. 0.49), while maintaining a lower false-positive rate (0.45). These differences indicate that VULTRIAL may provide a more conservative alternative to GPTLens, with improved balance across evaluation metrics. Overall, these findings suggest that VULTRIAL’s multi-agent reasoning approach can generalize to other models.

Evaluating Explanation. Agents in VULTRIAL provide explanations for their decisions, helping developers understand vulnerability classifications beyond a simple binary outcome. Because multiple agents contribute, developers may need to read the full multi-agent discussion to grasp the reasoning. To improve readability, we use GPT-4o to synthesize all agent reasoning into a single coherent explanation. We compare the synthesized VULTRIAL explanations with CoT-based ones using four metrics (i.e., completeness, informativeness, clarity, and actionability), each scored on a 1-5 scale [11, 53, 62, 67]. For fairness, we evaluate only cases where the P-C intersections of VULTRIAL and CoT match under GPT-4o. We conduct automated evaluation using a closed-source LLM from a different family (Claude-Haiku-4.5 [2]) and manual evaluation by three external annotators on 20 instances, without revealing explanation sources. Both evaluations show that synthesized VULTRIAL explanations outperform CoT across all metrics (Table 7). Generated explanations and evaluation results are publicly available [60].

6 Threats to Validity

External Validity. In this study, we focused the evaluation using PrimeVul pair data, which is a challenging recently-released vulnerability detection task on C/C++. We did not use other datasets as they may not be free from bias that have been uncovered by Ding et al. [12]. In the future, we plan to investigate VULTRIAL performance using other clean datasets. Another threat is the risk of data leakage, as ChatGPT’s training data is not publicly disclosed. We mitigate this by evaluating VULTRIAL using LLaMA-3.1-8B, whose training cutoff (Dec 2023) is before PrimeVul’s GitHub release (Mar 2024). VULTRIAL also identifies previously unknown real-world vulnerabilities, including CVE-2025-49000, showing it can generalize beyond training data. Therefore, we believe the threats are minimal.

Construct Validity. In this study, we use both pair-wise metrics and traditional metrics in our experiments (Section 3.4). These metrics provide a reliable measure of comparative performance, minimizing the threat to construct validity. In our experiments, VULTRIAL uses a single underlying model for all agents (e.g., GPT-4o and LLaMA-3.1-8B), we rely on prompt-based role specialization

Table 7: Results of automatic and manual evaluations of the generated explanation from CoT and synthesized VULTRIAL.

	CoT / Synthesized VULTRIAL			
	Completeness	Clarity	Actionability	Inform.
Claude [2]	3.15 / 4.10	3.60 / 3.85	2.15 / 4.00	2.80 / 3.95
Human	3.83 / 4.32	3.72 / 4.20	2.37 / 4.52	3.12 / 3.63

to induce diverse behaviors (e.g., prosecutor vs. defense attorney). Prior works [5, 26, 46, 56, 63] have shown that such prompt-induced roles can lead to qualitatively different reasoning paths even within the same model. In future work, we plan to use different LLMs for each agent role to further enhance diversity.

Internal Validity. Threats to internal validity in our study refer to possible errors in our experiment. To mitigate the risk, we have carefully checked the experiments that we run in our study. The code, prompt, and results are also included in our replication package [60]. Furthermore, we set the temperature to 0 to reduce the randomness effect of the experiment. We also did the preliminary evaluation where we ran the VULTRIAL on testing data three times and found that the standard deviation is low (0.7).

7 Related Work

Single-Agent Vulnerability Detection. Recent work has explored both DL-based and LLM-based methods for vulnerability detection. DL approaches, such as LineVul [18], achieve high recall with the use of transformer-based line-level classification. Ni et al. [36] proposed a semantic representation learning method, while Steenhoek et al. [47] incorporated dataflow. In contrast, LLM-based methods have gained traction for their strong results [25, 69]. Other studies have explored prompting and CoT [12, 37, 48, 50, 52, 65, 69]. Tamberg and Bahsi [50] benchmarked LLMs using advanced prompting techniques, including CoT, tree-of-thought, and self-consistency, finding that CoT with GPT-4 achieved the best performance and outperformed static analysis tools. Similarly, Zhou et al. [69] demonstrated that GPT-4 with prompting outperforms fine-tuned CodeBERT. Ullah et al. [52] evaluated commercial LLMs across various prompting strategies, including CoT, and found promising detection performance. Recent studies have further highlighted the limitations of CoT [37, 48, 65]. Notably, PrimeVul’s seminal work found that GPT-3.5 and GPT-4 perform poorly on PrimeVul’s pair test set, reinforcing concerns about CoT’s reliability in vulnerability detection. Efforts to enhance LLM-based methods include finetuning and retrieval-augmented generation (RAG). Shestov et al. [44] showed that finetuning boosts recall but is costly and dataset-sensitive. Du et al. [13] proposed Vul-RAG, which improves factual grounding via structured knowledge injection. Sun et al. [49] introduced LLM4Vuln, a framework that evaluates and decouples LLM reasoning capabilities. In this work, we use DL-based methods and CoT as baselines and show that VULTRIAL outperforms them on PrimeVul.

Multi-Agent Vulnerability Detection. Multi-agent frameworks can enhance LLM-based vulnerability detection by enabling collaboration and specialization among agents [23]. EvalSVA [58] introduces a multi-agent framework where agents act as security experts. Similarly, MuCoLD [33] agents assume roles such as tester and developer, which collaboratively analyze code. In contrast, VULTRIAL employs a structured multi-agent design inspired by a courtroom,

where agents take on distinct roles such as prosecutor, defender, and judge. This encourages adversarial debate and balanced reasoning that leads to robust decision-making. We excluded these recent works as baselines due to the lack of replication packages and insufficient prompt details to replicate the study.

Other multi-agent-based approaches have been explored for smart contract auditing [26, 32, 57]. LLM-SmartAudit [57] uses collaborative decision-making with role-play and reasoning to detect vulnerabilities in smart contracts. Similarly, iAudit [32] assigns specific roles, such as detector, reasoner, and critic, to guide the analysis of Solidity smart contracts. GPTLens [26] employs multiple LLM-based agents acting as independent prosecutors to identify vulnerabilities, with a critic agent reviewing and ranking the findings. Our work differs from these studies in two key ways. First, we focus on vulnerability detection in C and C++ on PrimeVul, a benchmark comprising challenging real-world examples. Second, VULTRIAL adopts a structured, courtroom-inspired framework specifically tailored for general-purpose languages beyond smart contracts. As these studies focus on smart contracts, their scope differs from our work for general-purpose programming languages. Nonetheless, we found that GPTLens can be adapted to general-purpose vulnerability detection and included it as a baseline in our experiments (Section 3.3). Our results demonstrate that VULTRIAL substantially outperforms GPTLens and single-agent baselines on PrimeVul.

8 Conclusion and Future Work

We proposed VULTRIAL, a novel explainable multi-agent LLM-based approach for vulnerability detection that applies a courtroom-style paradigm. VULTRIAL comprises four role-specific agents: the *security researcher* (prosecutor), *code author* (defense attorney), *moderator* (judge), and *review board* (jury). Each agent is essential to get the best performance (Section 4.4). We evaluated VULTRIAL against single-agent (CodeBERT [16], CodeT5 [55], UniXCoder [21], LineVul [18], and Ding et al.’s CoT [12]) and multi-agent (GPTLens [26]) baselines. Our results show that VULTRIAL outperforms both the single-agent and multi-agent baselines, with instruction tuning in the *moderator agent* further enhancing its performance. We have also demonstrated the efficacy of VULTRIAL across different LLMs, including an open-source, in-house-deployable model (LLaMA-3.1-8B), as well as the high quality of its generated explanations and its ability to uncover multiple zero-day vulnerabilities.

For future work, we plan to apply VULTRIAL to more datasets to evaluate its generalizability further. We also plan to consider the strengths and weaknesses of different LLMs to choose the one that best fits as the base model for each role-specific task.

Acknowledgments

This research / project is supported by the National Research Foundation, Singapore, and the Smart Nation Group under the Smart Nation Group’s Translational R&D Grant (Award No. TRANS2023-TGC02). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore or the Smart Nation Group.

References

- [1] Zahra Abbasiantaeb, Yifei Yuan, Evangelos Kanoulas, and Mohammad Aliannejadi. 2024. Let the llms talk: Simulating human-to-human conversational qa via zero-shot llm-to-llm interactions. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 8–17.
- [2] Anthropic. 2025. Claude 4.5 Haiku. <https://www.anthropic.com/claude/haiku>.
- [3] Apache Software Foundation. 2025. Apache Superset. <https://github.com/apache/superset>.
- [4] Saikat Chakraborty, Rahul Krishna, Yangruibo Ding, and Baishakhi Ray. 2021. Deep learning based vulnerability detection: Are we there yet? *IEEE Transactions on Software Engineering* 48, 9 (2021), 3280–3296.
- [5] Anwoy Chatterjee, HSVNS Kowndinya Renduchintala, Sumit Bhatia, and Tanmoy Chakraborty. 2024. Posix: A prompt sensitivity index for large language models. *arXiv preprint arXiv:2410.02185* (2024).
- [6] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. 2024. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=EHg5GDnyq1>
- [7] Yizheng Chen, Zhoujie Ding, Lamy ALOWAIN, Xinyun Chen, and David Wagner. 2023. Diversevul: A new vulnerable source code dataset for deep learning based vulnerability detection. In *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*. 654–668.
- [8] Xiaoxue Cheng, Junyi Li, Wayne Xin Zhao, and Ji-Rong Wen. 2024. Chainlm: Empowering large language models with improved chain-of-thought prompting. *The 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING)* (2024).
- [9] Boris Chernis and Rakesh Verma. 2018. Machine learning methods for software vulnerability detection. In *Proceedings of the fourth ACM international workshop on security and privacy analytics*. 31–39.
- [10] DataHub Contributors. 2025. DataHub. <https://github.com/datahub-project/datahub>.
- [11] Helmut Degen, Christof Budnik, Gregory Conte, Andrew Lintereur, and Seth Weber. 2022. How to explain it to energy engineers? A qualitative user study about trustworthiness, understandability, and actionability. In *International Conference on Human-Computer Interaction*. Springer, 262–284.
- [12] Yangruibo Ding, Yanjun Fu, Omniyyah Ibrahim, Chawin Sitawarin, Xinyun Chen, Basel Alomair, David Wagner, Baishakhi Ray, and Yizheng Chen. 2025. Vulnerability detection with code language models: How far are we? *ISSTA 2025* (2025).
- [13] Xueying Du, Geng Zheng, Kaixin Wang, Yi Zou, Yujia Wang, Wentai Deng, Jiayi Feng, Mingwei Liu, Bihuan Chen, Xin Peng, et al. 2024. Vul-rag: Enhancing llm-based vulnerability detection via knowledge-level rag. *arXiv preprint arXiv:2406.11147* (2024).
- [14] Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. 2023. Improving factuality and reasoning in language models through multiagent debate. In *Forty-first International Conference on Machine Learning*.
- [15] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv e-prints* (2024), arXiv–2407.
- [16] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).
- [17] Keith A Findley and Michael S Scott. 2006. The Multiple Dimensions of Tunnel Vision in Criminal Cases, 2006 Wis. L. Rev 291 (2006), 292.
- [18] Michael Fu and Chakrit Tantithamthavorn. 2022. Linevul: A transformer-based line-level vulnerability prediction. In *Proceedings of the 19th International Conference on Mining Software Repositories*. 608–620.
- [19] Chen Gao, Xiaochong Lan, Nian Li, Yuan Yuan, Jingtao Ding, Zhilun Zhou, Fengli Xu, and Yong Li. 2024. Large language models empowered agent-based modeling and simulation: A survey and perspectives. *Humanities and Social Sciences Communications* 11, 1 (2024), 1–24.
- [20] Brandon L Garrett. 2011. *Convicting the innocent: Where criminal prosecutions go wrong*. Harvard University Press.
- [21] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. Unixcoder: Unified cross-modal pre-training for code representation. *arXiv preprint arXiv:2203.03850* (2022).
- [22] Valerie P Hans, John H Blume, Theodore Eisenberg, Amelia Courtney Hritz, Sheri Lynn Johnson, Caisa Elizabeth Royer, and Martin T Wells. 2015. The death penalty: Should the judge or the jury decide who dies? *Journal of Empirical Legal Studies* 12, 1 (2015), 70–99.
- [23] Junda He, Christoph Treude, and David Lo. 2024. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* (2024).
- [24] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30.
- [25] Sihao Hu, Tiansheng Huang, Fatih Ilhan, Selim Furkan Tekin, and Ling Liu. 2022. Replication Package of GPTLens. <https://github.com/git-disl/GPTLens>.
- [26] Sihao Hu, Tiansheng Huang, Fatih Ilhan, Selim Furkan Tekin, and Ling Liu. 2023. Large language model-powered smart contract vulnerability detection: New perspectives. In *2023 5th IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*. IEEE, 297–306.
- [27] Zhe Hu, Hou Pong Chan, Jing Li, and Yu Yin. 2024. Unlocking Varied Perspectives: A Persona-Based Multi-Agent Framework with Debate-Driven Text Planning for Argument Generation. *arXiv e-prints* (2024), arXiv–2406.
- [28] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems* 43, 2 (2025), 1–55.
- [29] InvenTree Contributors. 2025. InvenTree. <https://github.com/inventree/InvenTree>.
- [30] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems* 35 (2022), 22199–22213.
- [31] Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujia Yang, Shuming Shi, and Zhaopeng Tu. 2023. Encouraging divergent thinking in large language models through multi-agent debate. *arXiv preprint arXiv:2305.19118* (2023).
- [32] Wei Ma, Daoyuan Wu, Yuqiang Sun, Tianwen Wang, Shangqing Liu, Jian Zhang, Yue Xue, and Yang Liu. 2024. Combining fine-tuning and llm-based agents for intuitive smart contract auditing with justifications. *arXiv preprint arXiv:2403.16073* (2024).
- [33] Zhenyao Mao, Jialong Li, Dongming Jin, Munan Li, and Kenji Tei. 2024. Multi-role consensus through llms discussions for vulnerability detection. In *2024 IEEE 24th International Conference on Software Quality, Reliability, and Security Companion (QRS-C)*. IEEE, 1318–1319.
- [34] Marah Stith McLeod. 2024. A Democratic Restraint on Incarceration. *Fla. L. Rev.* 76 (2024), 1147.
- [35] Alexander Tobias Neumann, Yue Yin, Sulayman Sowe, Stefan Decker, and Matthias Jarke. 2024. An llm-driven chatbot in higher education for databases and information systems. *IEEE Transactions on Education* (2024).
- [36] Chao Ni, Xin Yin, Kaiwen Yang, Dehai Zhao, Zhenchang Xing, and Xin Xia. 2023. Distinguishing look-alike innocent and vulnerable code by subtle semantic representation learning and explanation. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1611–1622.
- [37] Yu Nong, Mohammed Aldeen, Long Cheng, Hongxin Hu, Feng Chen, and Haipeng Cai. 2024. Chain-of-thought prompting of large language models for discovering and fixing software vulnerabilities. *arXiv preprint arXiv:2402.17230* (2024).
- [38] OpenAI. 2023. GPT-3.5. <https://platform.openai.com/docs/models/gpt-3.5-turbo>
- [39] OpenAI. 2023. GPT-4. <https://openai.com/index/gpt-4/>
- [40] OpenAI. 2024. Fine-Tuning GPT-4o. <https://openai.com/index/gpt-4o-fine-tuning/>
- [41] OpenAI. 2024. GPT-4o. <https://openai.com/index/hello-gpt-4o/>
- [42] Niklas Risse and Marcel Böhme. 2024. Uncovering the limits of machine learning for automatic vulnerability detection. In *33rd USENIX Security Symposium (USENIX Security 24)*. 4247–4264.
- [43] Rebecca Russell, Louis Kim, Lei Hamilton, Tomo Lazovich, Jacob Harer, Onur Ozdemir, Paul Ellingwood, and Marc McConley. 2018. Automated vulnerability detection in source code using deep representation learning. In *2018 17th IEEE international conference on machine learning and applications (ICMLA)*. IEEE, 757–762.
- [44] Aleksei Shestov, Rodion Levichev, Ravil Mussabayev, Evgeny Maslov, Pavel Zadorozhny, Anton Cheshkov, Rustam Mussabayev, Alymzhan Toleu, Gulmira Tolegen, and Alexander Krassovitskiy. 2025. Finetuning large language models for vulnerability detection. *IEEE Access* (2025).
- [45] Jinxin Shi, Jiabao Zhao, Xingjiao Wu, Ruyi Xu, Yuan-Hao Jiang, and Liang He. 2025. Mitigating reasoning hallucination through Multi-agent Collaborative Filtering. *Expert Systems with Applications* 263 (2025), 125723.
- [46] Noah Shinn, Federico Cassano, Beck Labash, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366> 1 (2023).
- [47] Benjamin Steenhoeck, Hongyang Gao, and Wei Le. 2024. Dataflow analysis-inspired deep learning for efficient vulnerability detection. In *Proceedings of the 46th IEEE/ACM international conference on software engineering*. 1–13.
- [48] Benjamin Steenhoeck, Md Mahbubur Rahman, Monoshi Kumar Roy, Mirza Sanjida Alam, Earl T Barr, and Wei Le. 2024. A Comprehensive Study of the Capabilities of Large Language Models for Vulnerability Detection. *arXiv preprint arXiv:2403.17218* (2024).
- [49] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Wei Ma, Lyuye Zhang, Yang Liu, and Yingjiu Li. 2024. Llm4vuln: A unified evaluation framework for decoupling and enhancing llms' vulnerability reasoning. *arXiv preprint arXiv:2401.16185*

- (2024).
- [50] Karl Tamberg and Hayretdin Bahsi. 2024. Harnessing Large Language Models for Software Vulnerability Detection: A Comprehensive Benchmarking Study. *arXiv preprint arXiv:2405.15614* (2024).
 - [51] Zixuan Tan, Jiayuan Zhou, Xing Hu, Shengyi Pan, Kui Liu, and Xin Xia. 2024. Similar but Patched Code Considered Harmful—The Impact of Similar but Patched Code on Recurring Vulnerability Detection and How to Remove Them. *arXiv preprint arXiv:2412.20740* (2024).
 - [52] Saad Ullah, Mingji Han, Saurabh Pujar, Hammond Pearce, Ayse Coskun, and Gianluca Stringhini. 2023. LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks. *arXiv preprint arXiv:2312.12575* (2023).
 - [53] Mythreyi Velmurugan, Chun Ouyang, Yue Xu, Renuka Sindhgatta, Bemali Wickramanayake, and Catarina Moreira. 2025. Developing guidelines for functionally-grounded evaluation of explainable artificial intelligence using tabular data. *Engineering Applications of Artificial Intelligence* 141 (2025), 109772.
 - [54] Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. 2023. Can language models solve graph problems in natural language? *Advances in Neural Information Processing Systems* 36 (2023), 30840–30861.
 - [55] Yue Wang, Weishi Wang, Shafiq Joty, and Steven CH Hoi. 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv preprint arXiv:2109.00859* (2021).
 - [56] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.
 - [57] Zhiyuan Wei, Jing Sun, Zijiang Zhang, Xianhao Zhang, Meng Li, and Zhe Hou. 2024. LLM-SmartAudit: Advanced Smart Contract Vulnerability Detection. *arXiv preprint arXiv:2410.09381* (2024).
 - [58] Xin-Cheng Wen, Jiaxin Ye, Cuiyun Gao, Lianwei Wu, and Qing Liao. 2024. EvalSVA: Multi-Agent Evaluators for Next-Gen Software Vulnerability Assessment. *arXiv preprint arXiv:2501.14737* (2024).
 - [59] Ratnadira Widyasari, David Lo, and Lizi Liao. 2024. Beyond ChatGPT: Enhancing Software Quality Assurance Tasks with Diverse LLMs and Validation Techniques. *arXiv preprint arXiv:2409.01001* (2024).
 - [60] Ratnadira Widyasari, Martin Weyssow, Ivana Clairine Irsan, Han Wei Ang, Frank Liauw, Eng Lieh Ouh, Lwin Khin Shar, Hong Jin Kang, and David Lo. 2025. Replication Package of VulTrial. <https://figshare.com/s/1514bc9a7aa64b46d94e>.
 - [61] Ratnadira Widyasari, Martin Weyssow, Ivana Clairine Irsan, Han Wei Ang, Frank Liauw, Eng Lieh Ouh, Lwin Khin Shar, Hong Jin Kang, and David Lo. 2025. Results of VulTrial Experiments in the Wild. <https://figshare.com/s/69e98e1a3432302e24c7>.
 - [62] Ratnadira Widyasari, Ting Zhang, Abir Bouraffa, Walid Maalej, and David Lo. 2025. Explaining explanations: An empirical study of explanations in code reviews. *ACM Transactions on Software Engineering and Methodology* 34, 6 (2025), 1–30.
 - [63] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2024. Autogen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*.
 - [64] Zhe Xiong and Weiyu Dong. 2024. VulD-CodeBERT: CodeBERT-Based Vulnerability Detection Model for C/C++ Code. In *2024 6th International Conference on Communications, Information System and Computer Engineering (CISCE)*. IEEE, 914–919.
 - [65] Chenyuan Zhang, Hao Liu, Jiutian Zeng, Kejing Yang, Yuhong Li, and Hui Li. 2024. Prompt-enhanced software vulnerability detection using chatgpt. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*. 276–277.
 - [66] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lema Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, et al. 2023. Siren's song in the ai ocean: A survey on hallucination in large language models. *arXiv preprint arXiv:2309.01219* 2, 5 (2023).
 - [67] Jianlong Zhou, Amir H Gandomi, Fang Chen, and Andreas Holzinger. 2021. Evaluating the quality of machine learning explanations: A survey on methods and metrics. *Electronics* 10, 5 (2021), 593.
 - [68] Xin Zhou, Sicong Cao, Xiaobing Sun, and David Lo. 2024. Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Transactions on Software Engineering and Methodology* (2024).
 - [69] Xin Zhou, Ting Zhang, and David Lo. 2024. Large language model for vulnerability detection: Emerging results and future directions. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. 47–51.
 - [70] Yuchen Zhuang, Yue Yu, Kuan Wang, Haotian Sun, and Chao Zhang. 2023. Toolqa: A dataset for llm question answering with external tools. *Advances in Neural Information Processing Systems* 36 (2023), 50117–50143.
 - [71] Deqing Zou, Sujuan Wang, Shouhuai Xu, Zhen Li, and Hai Jin. 2021. μ VulDeePecker: A Deep Learning-Based System for Multiclass Vulnerability Detection. *IEEE Transactions on Dependable and Secure Computing* 18, 05 (Sept. 2021), 2224–2236. doi:10.1109/TDSC.2019.2942930
 - [72] Deqing Zou, Yawei Zhu, Shouhuai Xu, Zhen Li, Hai Jin, and Hengkai Ye. 2021. Interpreting deep learning-based vulnerability detector predictions based on heuristic searching. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 30, 2 (2021), 1–31.